

Advanced Analysis: GSEA, Virulence, and Networks

López-Chablé Edgard Uriel

Contents

1. Environment Setup	1
2. Data Import	2
3. Gene Set Enrichment Analysis (GSEA)	2
4. Virulence Profiling	5
5. Upstream Regulators Analysis	8
6. Bioenergetics and Energy Targets	10
7. Network Analysis (PPI Hubs & Clusters)	10
Session Information	14

```
# Load necessary packages
library(clusterProfiler) # For enrichment analysis (GO/KEGG)
library(enrichplot)      # To plot enrichment results
library(tidyverse)       # For organizing and cleaning data
library(ggplot2)         # For custom plots
library(pheatmap)        # To create heatmaps
library(viridis)         # Color palettes
library(ggrepel)         # To prevent text overlap in plots
library(KEGGREST)        # To get pathway names from the KEGG API
library(STRINGdb)        # To get protein-protein interactions
library(igraph)          # To build and analyze networks
library(ggraph)          # To graph networks
library(tidygraph)       # To modify networks

# TIFF output directory (created automatically)
tiff_dir <- "../results/TIFFs/5_AdvancedAnalysis"
dir.create(tiff_dir, showWarnings = FALSE, recursive = TRUE)
```

1. Environment Setup

We create folders to keep things organized. Analyses like KEGG create many images that can quickly clutter the main folder.

```
adv_dir <- "../results/advanced_analysis"
dir.create(adv_dir, showWarnings = FALSE, recursive = TRUE)
```

2. Data Import

We load the ranked list (ordered by Log2FC) and the full database. GSEA needs the complete list of genes, not just the significant ones.

```
# Prepare the ranked list for GSEA
ranked_df <- readRDS("../results/advanced_analysis/gsea_ranked_list.rds") %>%
  # Keep only the highest log2FC value for duplicate genes
  group_by(GeneID) %>%
  slice_max(order_by = abs(log2FoldChange), n = 1, with_ties = FALSE) %>%
  ungroup()

gsea_vector <- ranked_df$log2FoldChange
names(gsea_vector) <- ranked_df$GeneID
gsea_vector <- sort(gsea_vector, decreasing = TRUE)

# Load full differential expression data
full_annotation <- read_csv("../results/advanced_analysis/full_annotated_results.csv")

## Rows: 4502 Columns: 28
## -- Column specification -----
## Delimiter: ","
## chr (20): ...1, GeneID, seed_ortholog, eggNOG_OGs, max_annot_lvl, COG_catego...
## dbl (8): baseMean, log2FoldChange, lfcSE, stat, pvalue, padj, evalute, score
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

# Rebuild the KEGG dictionary from our eggNOG file
eggnog_data <- read_delim("../data/annotation.tsv")

term2gene_kegg <- eggnog_data %>%
  dplyr::select(KEGG_Pathway, X.query) %>%
  tidyr::separate_rows(KEGG_Pathway, sep = ",") %>%
  dplyr::filter(grepl("map", KEGG_Pathway)) %>%
  dplyr::mutate(query = trimws(X.query), KEGG_Pathway = trimws(KEGG_Pathway)) %>%
  dplyr::select(KEGG_Pathway, X.query) %>%
  as.data.frame()
```

3. Gene Set Enrichment Analysis (GSEA)

GSEA looks at the whole ordered gene list to find if entire pathways are globally up-regulated or down-regulated.

```
# Get official KEGG pathway names from the web
kegg_names <- keggList("pathway")

# Clean up the names to make them easier to read
term2name_kegg <- data.frame(
  PathwayID = sub("path:", "", names(kegg_names)),
```

```

  PathwayName = as.character(kegg_names)
)
term2name_kegg$PathwayName <- gsub(" - Reference pathway", "", term2name_kegg$PathwayName)

# Run GSEA (seeded so that p-values are reproducible)
set.seed(124)
gsea_kegg <- GSEA(geneList = gsea_vector,
  TERM2GENE = term2gene_kegg,
  TERM2NAME = term2name_kegg,
  pvalueCutoff = 0.05,
  pAdjustMethod = "BH",
  minGSSize = 10,
  maxGSSize = 500,
  eps = 0,
  seed = TRUE) # reproducible permutations (uses set.seed above)

```

3.1. Systemic Shifts (Ridgeplot)

This plot shows how genes in the significant pathways are distributed.

```

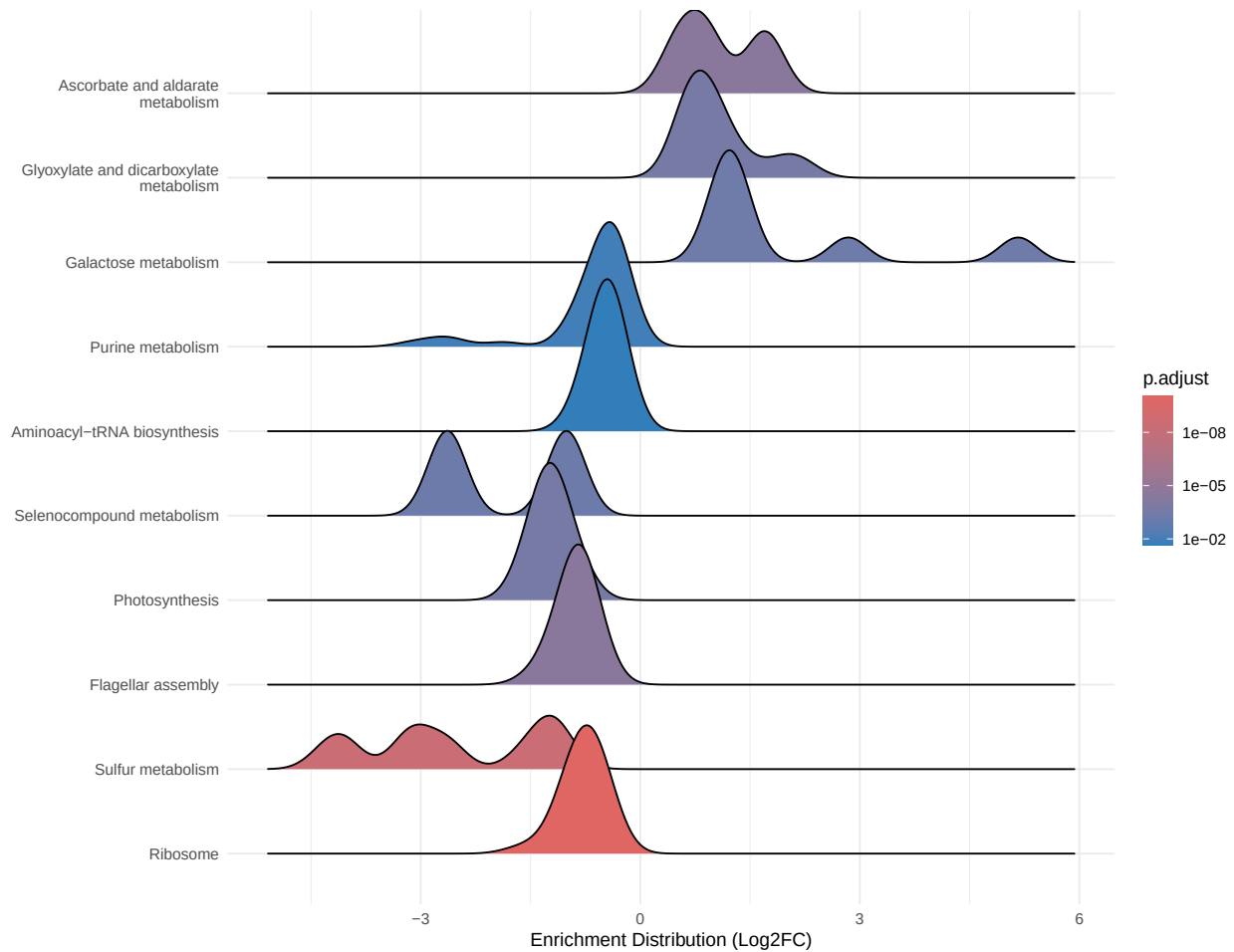
p_ridge <- ridgeplot(gsea_kegg, showCategory = 10) +
  labs(title = "GSEA Ridgeplot: Systemic KEGG Pathway Shifts",
    x = "Enrichment Distribution (Log2FC)") +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", size = 16))

p_ridge

```

Picking joint bandwidth of 0.257

GSEA Ridgeplot: Systemic KEGG Pathway Shifts



```
ggsave(filename = paste0(adv_dir, "/gsea_ridgeplot.pdf"), plot = p_ridge, width = 10, height = 8)
```

```
## Picking joint bandwidth of 0.257
```

```
ggsave(filename = file.path(tiff_dir, "gsea_ridgeplot.tiff"), plot = p_ridge, width = 10, height = 8, dpi = 300)
```

```
## Picking joint bandwidth of 0.257
```

3.2. Top Enriched Pathway Plot

```
# Pick a specific pathway (e.g., the second one) to avoid generic ones like Ribosome
target_pathway_idx <- 2
top_gsea_pathway <- gsea_kegg@result$ID[target_pathway_idx]

p_gseaplot <- gseaplot2(gsea_kegg,
  geneSetID = top_gsea_pathway,
  title = paste("GSEA - Pathway:", gsea_kegg@result$Description[target_pathway_idx]),
  pvalue_table = TRUE)
```

```
pdf(paste0(adv_dir, "/target_pathway_gseaplot.pdf"), width = 10, height = 8.5)
print(p_gseaplot)
dev.off()

## pdf
## 2

tiff(file.path(tiff_dir, "target_pathway_gseaplot.tiff"), width = 10, height = 8.5,
      units = "in", res = 300, compression = "lzw")
print(p_gseaplot)
dev.off()

## pdf
## 2
```

4. Virulence Profiling

We filter the data to find plant cell wall degrading enzymes and secretion systems to see how the pathogen's attack tools change.

```
# Keywords to find virulence factors.
# Functional keywords are searched in the eggNOG description; the gene-family
# prefixes (pel, peh, out, kdg) are matched only at the start of the gene name,
# so that e.g. "out" does not capture every "outer membrane" protein.
virulence_desc_kw <- "pectate lyase|cellulase|protease|secretion system"
virulence_name_kw <- "^(pel|peh|out|kdg)"

virulence_df <- full_annotation %>%
  filter(padj < 0.05) %>%
  filter(str_detect(tolower(Description), virulence_desc_kw) |
         str_detect(tolower(Preferred_name), virulence_name_kw)) %>%
  arrange(desc(abs(log2FoldChange)))

virulence_genes <- virulence_df$GeneID
message("Found ", length(virulence_genes), " significant virulence factors.")

## Found 45 significant virulence factors.

write_csv(virulence_df, paste0(adv_dir, "/virulence_factors_table.csv"))
```

4.1. Virulence Factors Heatmap

```
# Load normalized counts and metadata
load(file = "../results/dds_norm.RData")
metadatos <- read.csv(file = "../data/metadata_all.csv", row.names = 1)
```

```

colnames(normcounts) <- metadatos$Replicates

# Fix row names to match our GeneIDs
rownames(normcounts) <- str_extract(rownames(normcounts), "WP_[0-9]+\.[0-9]+")
normcounts <- normcounts[!is.na(rownames(normcounts)), ]

# Keep only the virulence genes
virulence_matrix <- normcounts[virulence_genes, ]

# Use common gene names (like pelA) if we have them
rownames(virulence_matrix) <- ifelse(!is.na(virulence_df$Preferred_name),
                                     virulence_df$Preferred_name,
                                     virulence_df$GeneID)

p_virulence_hm <- pheatmap(virulence_matrix,
                           scale = "row",
                           color = viridis(100),
                           treeheight_row = 15,
                           treeheight_col = 15,
                           main = "Pectobacterium Virulence Factors Expression",
                           fontsize_row = 8)

```



```
ggsave(filename = paste0(adv_dir, "/heatmap_virulence.pdf"), plot = p_virulence_hm, width = 8, height = 10)
ggsave(filename = file.path(tiff_dir, "heatmap_virulence.tiff"), plot = p_virulence_hm, width = 8, height = 10)
```

5. Upstream Regulators Analysis

We search for transcription factors and DNA-binding proteins that might be controlling these changes.

```
tf_keywords <- "transcription factor|regulator|dna-binding" # matched against lower-cased text

# Four-category coloring: background / NS regulator / up-regulator / down-regulator
volcano_data <- full_annotation %>%
  mutate(
    Is_Regulator = if_else(
      str_detect(tolower(Description), tf_keywords) |
      str_detect(tolower(Preferred_name), tf_keywords), "Yes", "No"),
    Category = case_when(
      Is_Regulator == "Yes" & !is.na(padj) & padj < 0.05 & log2FoldChange > 1 ~ "Regulator ↑ in Treatment",
      Is_Regulator == "Yes" & !is.na(padj) & padj < 0.05 & log2FoldChange < -1 ~ "Regulator ↓ in Treatment",
      Is_Regulator == "Yes" ~ "Regulator (NS)",
      TRUE ~ "Background"
    ),
    Label = if_else(
      Is_Regulator == "Yes" & !is.na(padj) & padj < 0.05 & abs(log2FoldChange) > 1,
      coalesce(
        na_if(Preferred_name, "-"), # use Preferred_name when available
        str_extract(GeneID, "WP_[0-9]+\\.[0-9]+"), # else short WP_id
        GeneID # last resort: full id
      ),
      NA_character_ # NA so ggrepel truly ignores non-labeled points
    )
  )

# Palette: red = higher in Treatment (pos FC), blue = higher in Control (neg FC)
reg_colors <- c("Background" = "#BDBDBD",
               "Regulator (NS)" = "#9ECAE1",
               "Regulator ↑ in Treatment" = "#D73027",
               "Regulator ↓ in Treatment" = "#4575B4")
reg_sizes <- c("Background"=1.2, "Regulator (NS)"=2.5,
               "Regulator ↑ in Treatment"=3.5, "Regulator ↓ in Treatment"=3.5)
reg_alpha <- c("Background"=0.35, "Regulator (NS)"=0.65,
               "Regulator ↑ in Treatment"=1, "Regulator ↓ in Treatment"=1)

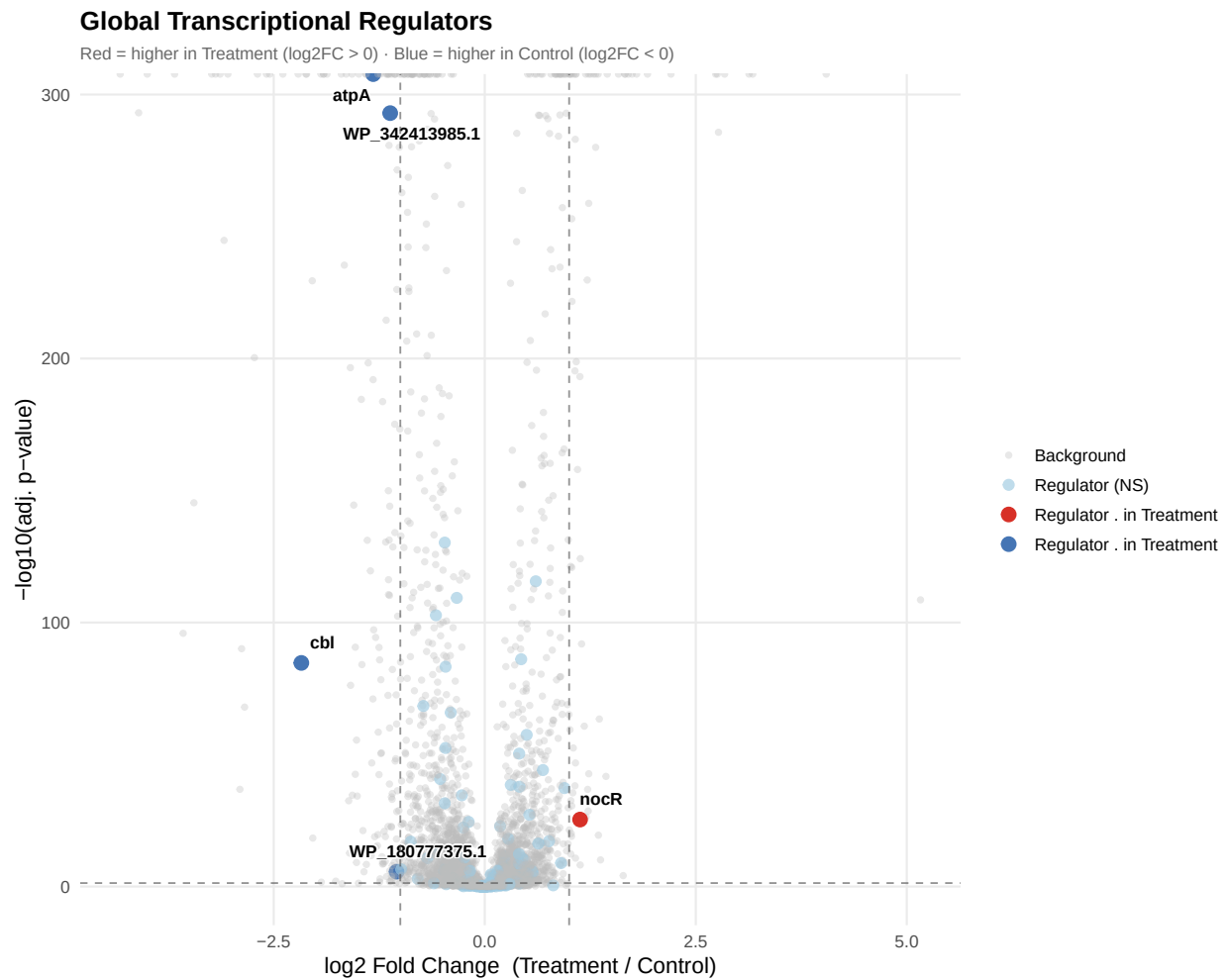
p_tf_volcano <- ggplot(volcano_data, aes(x = log2FoldChange, y = -log10(padj))) +
  geom_point(aes(color = Category, size = Category, alpha = Category)) +
  scale_color_manual(values = reg_colors, name = NULL) +
  scale_size_manual(values = reg_sizes, name = NULL) +
  scale_alpha_manual(values = reg_alpha, name = NULL) +
  geom_text_repel(data = ~ filter(.x, !is.na(Label)),
                 aes(label = Label),
                 size = 3.5, fontface = "bold",
                 box.padding = 0.5, point.padding = 0.3,
                 max.overlaps = Inf,
                 color = "black", bg.color = "white", bg.r = 0.12,
                 segment.color = "grey50", segment.size = 0.4) +
  geom_vline(xintercept = c(-1, 1), linetype = "dashed",
             color = "grey30", alpha = 0.6, linewidth = 0.5) +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed",
             color = "grey30", alpha = 0.6, linewidth = 0.5) +
```

```

labs(title = "Global Transcriptional Regulators",
     subtitle = "Red = higher in Treatment (log2FC > 0) · Blue = higher in Control (log2FC < 0)",
     x = "log Fold Change (Treatment / Control)",
     y = "-log (adj. p-value)") +
theme_minimal(base_size = 13) +
theme(plot.title = element_text(face = "bold", size = 15),
     plot.subtitle = element_text(size = 10, color = "grey40"),
     panel.grid.minor = element_blank(),
     legend.position = "right")

```

p_tf_volcano



```

ggsave(filename = paste0(adv_dir, "/volcano_regulators.pdf"), plot = p_tf_volcano, width = 10, height = 10)
ggsave(filename = file.path(tiff_dir, "volcano_regulators.tiff"), plot = p_tf_volcano, width = 10, height = 10)

```

6. Bioenergetics and Energy Targets

We look for genes related to energy, like pumps and electron transport, which can be useful for lab experiments.

```
electro_keywords <- "atp synthase|efflux|pump|ion channel|cytochrome|electron transport|nadh dehydrogenase"

electro_genes <- full_annotation %>%
  filter(padj < 0.05) %>%
  filter(str_detect(tolower(Description), electro_keywords) |
         str_detect(tolower(Preferred_name), electro_keywords) |
         str_detect(tolower(GOs), "ion transport|atp biosynthetic")) %>%
  arrange(desc(log2FoldChange))

message("Found ", nrow(electro_genes), " bioenergetic targets.")
```

```
## Found 81 bioenergetic targets.
```

```
write_csv(electro_genes, paste0(adv_dir, "/electrochemistry_targets.csv"))
```

7. Network Analysis (PPI Hubs & Clusters)

We build a Protein-Protein Interaction (PPI) network using STRING's mapping tool. We apply a High-Confidence threshold (Score > 700) to remove noise and isolate distinct functional modules.

```
# 1. Prepare data for STRING mapping
deg_network <- full_annotation %>%
  filter(padj < 0.05) %>%
  dplyr::select(GeneID, Preferred_name, seed_ortholog, log2FoldChange) %>%
  mutate(search_name = coalesce(Preferred_name, GeneID)) %>%
  group_by(search_name) %>%
  slice_max(order_by = abs(log2FoldChange), n = 1, with_ties = FALSE) %>%
  ungroup()

# 2. Get Taxonomy ID and start STRINGdb (HIGH CONFIDENCE: 700)
valid_orthologs <- deg_network$seed_ortholog[!is.na(deg_network$seed_ortholog) & deg_network$seed_ortholog != ""]
tax_id <- as.numeric(strsplit(valid_orthologs[1], "\\.")[[1]][1])

string_db <- STRINGdb$new(version = "11.5", species = tax_id, score_threshold = 700)

# 3. Map genes securely
mapped_genes <- string_db$map(as.data.frame(deg_network), "search_name", removeUnmappedRows = TRUE)

## Warning: we couldn't map to STRING 64% of your identifiers

# 4. Get interactions
interactions <- string_db$get_interactions(mapped_genes$STRING_id)

# 5. Build and clean the network graph
```

```

ppi_graph <- graph_from_data_frame(d = interactions, directed = FALSE)
ppi_graph <- simplify(ppi_graph)

# 6. Process topology and extract Bottleneck Hubs
if (vcount(ppi_graph) > 0) {
  # Find clusters
  # Louvain is stochastic: without a seed, module membership (and therefore the
  # per-module hub selection below) varies between runs. Seed 124 reproduces the
  # hub set shown in the published figure; 13 of the 15 hubs are stable across seeds.
  set.seed(124)
  communities <- cluster_louvain(ppi_graph)
  ppi_graph <- set_vertex_attr(ppi_graph, "cluster", value = as.numeric(membership(communities)))

  # Calculate importance metrics
  ppi_graph <- set_vertex_attr(ppi_graph, "degree", value = degree(ppi_graph))
  ppi_graph <- set_vertex_attr(ppi_graph, "betweenness", value = betweenness(ppi_graph))

  # Add names back to nodes
  node_metadata <- data.frame(STRING_id = V(ppi_graph)$name) %>%
    left_join(mapped_genes, by = "STRING_id") %>%
    distinct(STRING_id, .keep_all = TRUE)

  ppi_graph <- set_vertex_attr(ppi_graph, "label", value = node_metadata$search_name)
  ppi_graph <- set_vertex_attr(ppi_graph, "log2FC", value = node_metadata$log2FoldChange)

  # STRATEGY CHANGE: Extract hubs based on Betweenness (Bottlenecks)
  # This labels critical bridges connecting different biological pathways.
  top_hubs <- node_metadata %>%
    mutate(Degree = V(ppi_graph)$degree,
           Betweenness = V(ppi_graph)$betweenness,
           Cluster = V(ppi_graph)$cluster) %>%
    group_by(Cluster) %>%
    slice_max(order_by = Betweenness, n = 3, with_ties = FALSE) %>%
    ungroup() %>%
    arrange(desc(Betweenness)) %>%
    head(15)

  write_csv(top_hubs, paste0(adv_dir, "/ppi_bottleneck_hubs.csv"))
} else {
  message("STRINGdb could not find any interactions above the 700 threshold.")
}

```

7.1. Network Visualization (ggraph)

We use `ggraph` with a Kamada-Kawai layout to separate distinct functional modules, while `ggrepel` highlights the bottleneck hubs identified above.

```

library(ggraph)
library(tidygraph)
library(magick)

if (vcount(ppi_graph) > 0) {
  net_components <- components(ppi_graph)

```

```

if (max(net_components$csizes) > 1) {
  main_component <- induced_subgraph(ppi_graph, vids = which(net_components$membership == which.max(n
  tidy_net <- as_tbl_graph(main_component) %>%
    mutate(
      is_hub          = name %in% top_hubs$STRING_id,
      hub_label       = ifelse(is_hub, label, NA),
      clean_betweenness = replace_na(betweenness, 0),
      max_bet         = max(clean_betweenness, na.rm = TRUE),
      node_size       = ifelse(max_bet > 0, (clean_betweenness / max_bet) * 10 + 3, 5)
    )

  p_net <- ggraph(tidy_net, layout = "kk") +
    geom_edge_link(color = "grey85", width = 0.5, alpha = 0.6) +
    geom_node_point(aes(size = node_size, color = as.factor(cluster), alpha = is_hub)) +
    scale_alpha_manual(values = c("TRUE" = 1, "FALSE" = 0.4), guide = "none") +
    geom_node_text(aes(label = hub_label),
      repel          = TRUE,
      fontface       = "bold",
      size           = 5,
      box.padding    = 1,
      point.padding  = 0.5,
      max.overlaps   = Inf,
      bg.color       = "white",
      bg.r           = 0.15,
      segment.color  = "grey40",
      segment.size   = 0.6) +
    scale_color_viridis_d(option = "viridis") +
    scale_size_continuous(range = c(1.5, 11), guide = "none") +
    labs(title      = "Protein-Protein Interaction Network",
      subtitle     = "Top 15 Bottleneck Hubs Highlighted (High Confidence STRING Score > 700)") +
    theme_void() +
    theme(legend.position = "none",
      plot.title      = element_text(face = "bold", size = 18, hjust = 0.5),
      plot.subtitle   = element_text(size = 14, hjust = 0.5, color = "grey40"))

  # Render ONCE to PDF, then convert to TIFF with magick (avoids triple-rendering)
  pdf_path <- paste0(adv_dir, "/ppi_high_confidence_network.pdf")
  tiff_path <- file.path(tiff_dir, "ppi_high_confidence_network.tiff")

  pdf(pdf_path, width = 12, height = 12)
  print(p_net)
  dev.off()

  image_write(
    image_read_pdf(pdf_path, density = 300),
    path = tiff_path, format = "tiff", compression = "lzw"
  )

  print(p_net)
} else {
  message("No connected components found to draw the plot.")
}

```

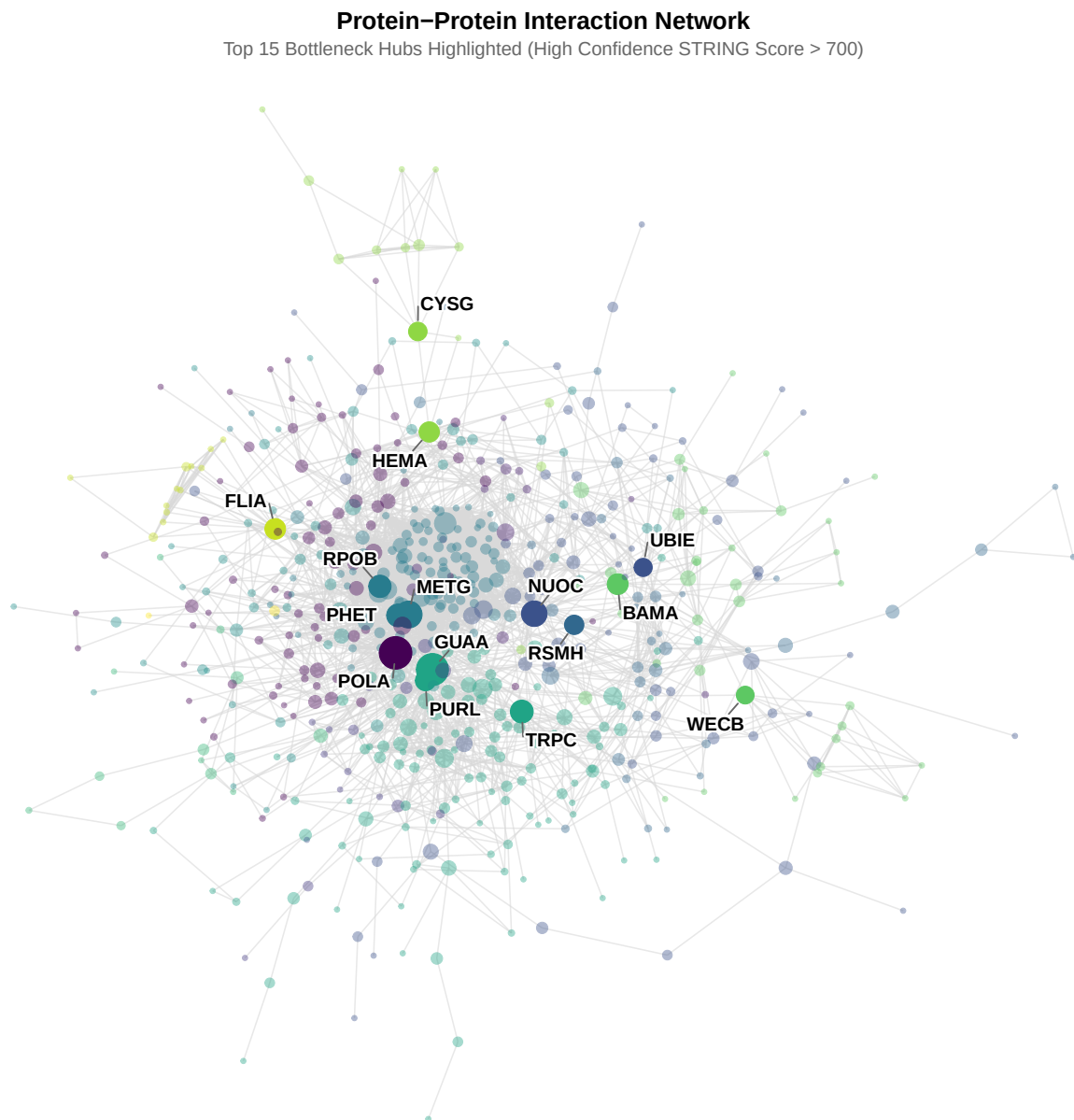
```

} else {
  message("Plot cancelled: the network is empty.")
}

```

```
## PDF error: No display font for 'Symbol'
```

```
## PDF error: No display font for 'ArialUnicode'
```



Session Information

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Spanish_Mexico.utf8  LC_CTYPE=Spanish_Mexico.utf8
## [3] LC_MONETARY=Spanish_Mexico.utf8 LC_NUMERIC=C
## [5] LC_TIME=Spanish_Mexico.utf8
##
## time zone: America/Mexico_City
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] tidygraph_1.3.1          ggraph_2.2.2
## [3] igraph_2.2.2             KEGGREST_1.50.0
## [5] magick_2.9.1            STRINGdb_2.22.0
## [7] UpSetR_1.4.1            GO.db_3.22.0
## [9] AnnotationDbi_1.72.0     pathview_1.50.0
## [11] enrichplot_1.30.5       clusterProfiler_4.18.4
## [13] viridis_0.6.5           viridisLite_0.4.3
## [15] pheatmap_1.0.13         cowplot_1.2.0
## [17] lubridate_1.9.5         forcats_1.0.1
## [19] stringr_1.6.0           dplyr_1.2.0
## [21] purrr_1.2.1            readr_2.2.0
## [23] tidyr_1.3.2            tibble_3.3.1
## [25] tidyverse_2.0.0         EnhancedVolcano_1.28.2
## [27] ggrepel_0.9.7           ggplot2_4.0.3
## [29] DESeq2_1.50.2           SummarizedExperiment_1.40.0
## [31] Biobase_2.70.0          MatrixGenerics_1.22.0
## [33] matrixStats_1.5.0       GenomicRanges_1.62.1
## [35] Seqinfo_1.0.0           IRanges_2.44.0
## [37] S4Vectors_0.48.0       BiocGenerics_0.56.0
## [39] generics_0.1.4
##
## loaded via a namespace (and not attached):
## [1] splines_4.5.2           bitops_1.0-9           ggplotify_0.1.3
## [4] R.oo_1.27.1            polyclip_1.10-7        graph_1.88.1
## [7] XML_3.99-0.23          lifecycle_1.0.5        lattice_0.22-7
## [10] vroom_1.7.1            MASS_7.3-65            magrittr_2.0.4
## [13] rmarkdown_2.31         plotrix_3.8-14         yaml_2.3.12
## [16] otl_0.2.0              ggtangle_0.1.2         askpass_1.2.1
## [19] DBI_1.3.0              RColorBrewer_1.1-3     abind_1.4-8
```

## [22] R.utils_2.13.0	RCurl_1.98-1.19	hash_2.2.6.4
## [25] yulab.utils_0.2.4	tweenr_2.0.3	rappdirs_0.3.4
## [28] gdttools_0.5.0	tidytree_0.4.8	proto_1.0.0
## [31] codetools_0.2-20	DelayedArray_0.36.1	pdftools_3.9.0
## [34] DOSE_4.4.0	ggforce_0.5.0	tidyselect_1.2.1
## [37] aplot_0.3.1	farver_2.1.2	jsonlite_2.0.0
## [40] ggribes_0.5.7	systemfonts_1.3.2	chron_2.3-63
## [43] tools_4.5.2	ggnewscale_0.5.2	treeio_1.34.0
## [46] ragg_1.5.0	snow_0.4-4	Rcpp_1.1.1
## [49] glue_1.8.0	gridExtra_2.3.1	SparseArray_1.10.10
## [52] xfun_0.55	qvalue_2.42.0	withr_3.0.3
## [55] fastmap_1.2.0	caTools_1.18.4	digest_0.6.39
## [58] timechange_0.4.0	R6_2.6.1	gridGraphics_0.5-1
## [61] qpdf_1.4.1	textshaping_1.0.4	gtools_3.9.5
## [64] RSQLite_2.4.6	R.methodsS3_1.8.2	utf8_1.2.6
## [67] fontLiberation_0.1.0	data.table_1.18.2.1	graphlayouts_1.2.5
## [70] httr_1.4.8	htmlwidgets_1.6.4	S4Arrays_1.10.1
## [73] scatterpie_0.2.6	sqldf_0.4-12	pkgconfig_2.0.3
## [76] gtable_0.3.6	blob_1.3.0	S7_0.2.1
## [79] XVector_0.50.0	htmltools_0.5.9	fontBitstreamVera_0.1.1
## [82] fgsea_1.36.2	scales_1.4.0	png_0.1-9
## [85] ggfun_0.2.1	knitr_1.51	tzdb_0.5.0
## [88] reshape2_1.4.5	curl_7.0.0	nlme_3.1-168
## [91] org.Hs.eg.db_3.22.0	cachem_1.1.0	KernSmooth_2.23-26
## [94] parallel_4.5.2	pillar_1.11.1	grid_4.5.2
## [97] vctrs_0.7.1	gplots_3.3.0	tidydr_0.0.6
## [100] cluster_2.1.8.1	Rgraphviz_2.54.0	evaluate_1.0.5
## [103] KEGGgraph_1.70.0	gsubfn_0.7	tinytex_0.60
## [106] cli_3.6.5	locfit_1.5-9.12	compiler_4.5.2
## [109] rlang_1.3.0	crayon_1.5.3	labeling_0.4.3
## [112] plyr_1.8.9	fs_2.1.0	ggiraph_0.9.6
## [115] stringi_1.8.7	BiocParallel_1.44.0	Biostrings_2.78.0
## [118] lazyeval_0.2.2	GOSemSim_2.36.0	fontquiver_0.2.1
## [121] Matrix_1.7-4	hms_1.1.4	patchwork_1.3.2
## [124] bit64_4.6.0-1	memoise_2.0.1	ggtree_4.0.5
## [127] fastmatch_1.1-8	bit_4.6.0	ape_5.8-1
## [130] gson_0.2.0		